AOOP PRACTICAL-1

2520030003

Kunamaneni Bhavana

Section-4

Code:

```
import java.time.LocalDateTime;
import java.util.UUID;
// Money Class
class Money {
    private final long amountInPaise;
    private final String currency;
    // Constructor with validation
    public Money(long amountInPaise, String currency) {
        if (amountInPaise < 0) {
            throw new IllegalArgumentException("Amount cannot be negative");
        }
        if (currency == null || currency.isEmpty()) {
            throw new IllegalArgumentException("Invalid currency");
        }
        this.amountInPaise = amountInPaise;
        this.currency = currency;
    }
    // Add
    public Money add(Money other) {
```

```
    checkCurrency(other);
    return new Money(
        Math.addExact(this.amountInPaise, other.amountInPaise),
        currency
    );
}
// Subtract
public Money subtract(Money other) {
    checkCurrency(other);

    if (this.amountInPaise < other.amountInPaise) {
        throw new IllegalArgumentException("Insufficient amount");
    }

    return new Money(
        this.amountInPaise - other.amountInPaise,
        currency
    );
}

// Check positive
public boolean isPositive() {
    return amountInPaise > 0;
}

private void checkCurrency(Money other) {
    if (other == null || !currency.equals(other.currency)) {
```

```
            throw new IllegalArgumentException("Currency mismatch");
        }
    }

    public long getAmountInPaise() {
        return amountInPaise;
    }

    public String getCurrency() {
        return currency;
    }

    @Override
    public String toString() {
        return currency + " " + (amountInPaise / 100.0);
    }
}

// Account Status Enum
enum AccountStatus {
    ACTIVE,
    FROZEN,
    CLOSED
}

// BankAccount Class
class BankAccount {
```

```
private final String accountNumber;
private Money balance;
private final Money overdraftLimit;
private AccountStatus status;

// Constructor
public BankAccount(String accountNumber, Money overdraftLimit) {
    this.accountNumber = accountNumber;
    this.overdraftLimit = overdraftLimit;
    this.balance = new Money(0, overdraftLimit.getCurrency());
    this.status = AccountStatus.ACTIVE;
}

// Deposit
public void deposit(Money amount) {
    checkActive();

    if (!amount.isPositive()) {
        throw new IllegalArgumentException("Deposit must be positive");
    }

    balance = balance.add(amount);
    System.out.println("Deposited: " + amount);
}

// Withdraw
```

```
public void withdraw(Money amount) {
    checkActive();

    if (!amount.isPositive()) {
        throw new IllegalArgumentException("Withdrawal must be positive");
    }

    Money available = balance.add(overdraftLimit);

    if (available.getAmountInPaise() < amount.getAmountInPaise()) {
        throw new IllegalArgumentException("Insufficient funds");
    }

    balance = new Money(
        Math.subtractExact(balance.getAmountInPaise(),
                           amount.getAmountInPaise()),
        balance.getCurrency()
    );

    System.out.println("Withdrawn: " + amount);
}

// Freeze
public void freeze() {
    checkActive();
    status = AccountStatus.FROZEN;
    System.out.println("Account frozen");
```

```
}

// Unfreeze
public void unfreeze() {
    if (status != AccountStatus.FROZEN) {
        throw new IllegalStateException("Account is not frozen");
    }

    status = AccountStatus.ACTIVE;
    System.out.println("Account unfrozen");
}

// Close
public void close() {
    if (status == AccountStatus.CLOSED) {
        throw new IllegalStateException("Account already closed");
    }

    status = AccountStatus.CLOSED;
    System.out.println("Account closed");
}

// Get Balance
public Money getBalance() {
    return balance;
}
```

```
    public String getAccountNumber() {
        return accountNumber;
    }

    public AccountStatus getStatus() {
        return status;
    }

    private void checkActive() {
        if (status != AccountStatus.ACTIVE) {
            throw new IllegalStateException(
                "Account is not active: " + status
            );
        }
    }
}

// LedgerEntry Class
class LedgerEntry {

    private final String transactionId;
    private final String transactionType;
    private final Money amount;
    private final String timestamp;

    // Constructor
    public LedgerEntry(String transactionType, Money amount) {
```

```
    this.transactionId = UUID.randomUUID().toString();
    this.transactionType = transactionType;
    this.amount = amount;
    this.timestamp = LocalDateTime.now().toString();
}

// Getters
public String getTransactionId() {
    return transactionId;
}

public String getTransactionType() {
    return transactionType;
}

public Money getAmount() {
    return amount;
}

public String getTimestamp() {
    return timestamp;
}

public void display() {
    System.out.println("Transaction ID: " + transactionId);
    System.out.println("Type: " + transactionType);
    System.out.println("Amount: " + amount);
```

```
        System.out.println("Timestamp: " + timestamp);
    }
}

// FundTransfer Class
class FundTransfer {

    private final String transferId;
    private final BankAccount source;
    private final BankAccount destination;
    private final Money amount;

    // Constructor
    public FundTransfer(BankAccount source,
                        BankAccount destination,
                        Money amount) {

        this.transferId = UUID.randomUUID().toString();
        this.source = source;
        this.destination = destination;
        this.amount = amount;
    }

    // Transfer
    public void transfer() {

        if (source == destination) {
```

```
    throw new IllegalArgumentException(
        "Source and destination cannot be the same"
    );
}

if (!amount.isPositive()) {
    throw new IllegalArgumentException(
        "Transfer amount must be positive"
    );
}

// Validate destination before withdrawing
if (destination.getStatus() != AccountStatus.ACTIVE) {
    throw new IllegalStateException(
        "Destination account is not active"
    );
}

// Withdraw from source
source.withdraw(amount);

// Deposit into destination
destination.deposit(amount);

System.out.println("Fund transfer successful!");
System.out.println("Transfer ID: " + transferId);
```

```
        LedgerEntry entry =
            new LedgerEntry("FUND TRANSFER", amount);

        entry.display();
    }
}

// Main Class
public class Main {

    public static void main(String[] args) {

        Money limit = new Money(500000, "INR");

        BankAccount a1 = new BankAccount("1001", limit);
        BankAccount a2 = new BankAccount("1002", limit);

        System.out.println("----- DEPOSIT -----");

        a1.deposit(new Money(1000000, "INR"));

        System.out.println("Balance: " + a1.getBalance());

        System.out.println("\n----- WITHDRAW -----");

        a1.withdraw(new Money(200000, "INR"));
```

```
System.out.println("Balance: " + a1.getBalance());

System.out.println("\n----- FUND TRANSFER -----");

FundTransfer ft = new FundTransfer(
    a1,
    a2,
    new Money(300000, "INR")
);

ft.transfer();

System.out.println("\n----- FINAL BALANCES -----");

System.out.println("Account 1001: " + a1.getBalance());
System.out.println("Account 1002: " + a2.getBalance());

System.out.println("\n----- FREEZE ACCOUNT -----");

a1.freeze();

System.out.println("Status: " + a1.getStatus());

System.out.println("\n----- UNFREEZE ACCOUNT -----");

a1.unfreeze();
```

```
        System.out.println("Status: " + a1.getStatus());

        System.out.println("\n----- CLOSE ACCOUNT -----");

        a1.close();

        System.out.println("Status: " + a1.getStatus());
    }
}
```

Output:



